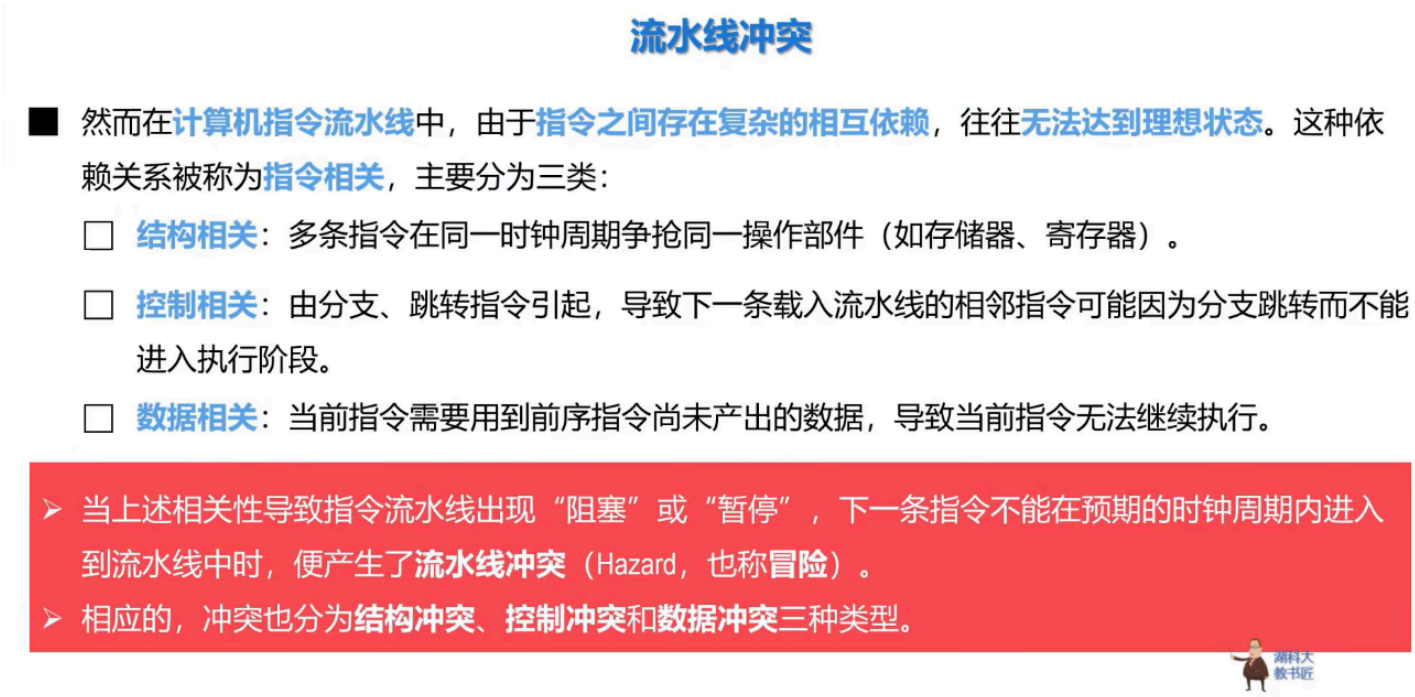
6.29 指令流水线——流水线冲突解决（结构与控制冲突的解决）

考纲定位：计算机组成原理 · 第 6 章 中央处理器（6.29） | 重要度 ★★★★ | 常见考法：选择题（结构冲突、控制冲突、分支预测、延迟槽、冲刷）

一句话核心：结构冲突争资源，控制冲突争“下一条取哪”；前者靠资源重复/分时，后者靠预测/延迟/冲刷

一、流水线冲突的定义



本图定位：三类冲突的总览——结构、数据、控制

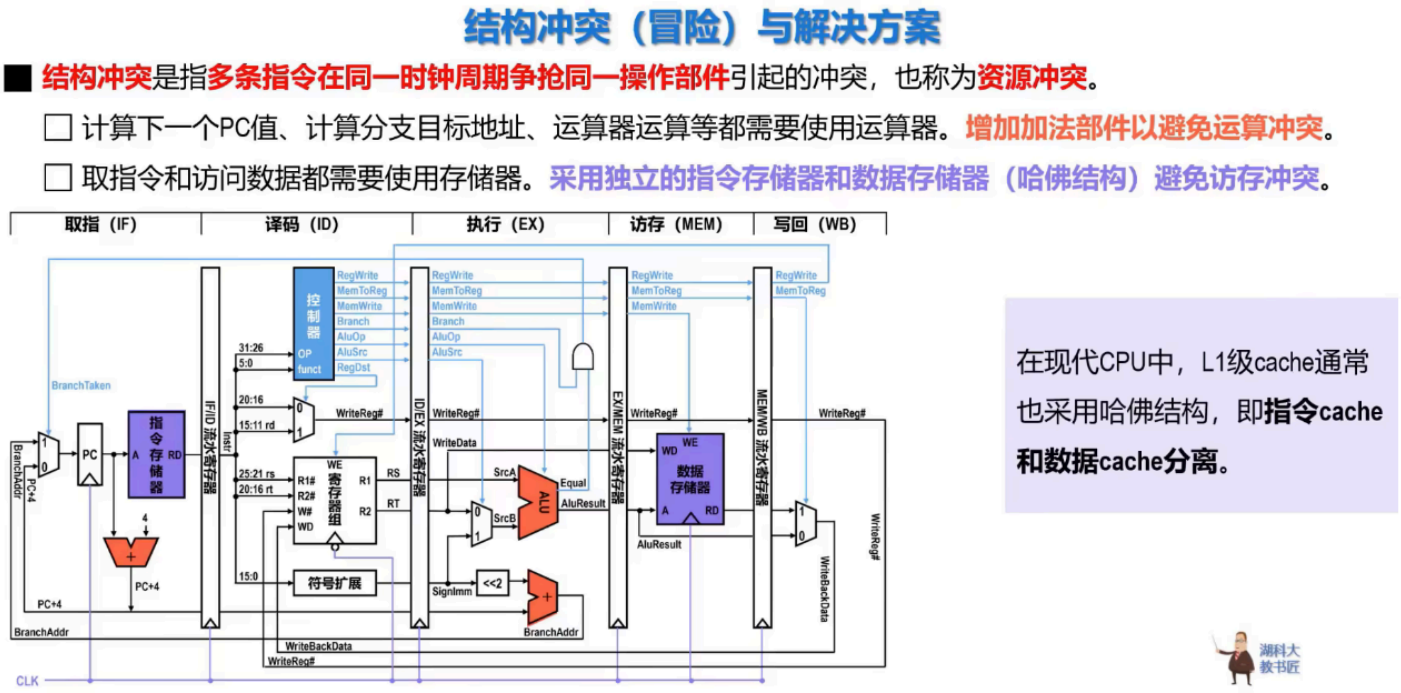
考点/易错

- 冲突（冒险）= 指令间依赖导致下一条指令不能在预期时钟进入流水线
- 结构冲突：争用同一部件/资源
- 数据冲突：后一条要用前一条尚未产生的结果（RAW）
- 控制冲突：分支/跳转导致后续取指可能作废
- 易错：三类冲突的成因不同，解决方法也不同，不要混

补充

- 408 常问“哪类冲突由分支引起”：控制冲突；“哪类冲突由指令与数据共用存储器引起”：结构冲突
- 数据冲突的详细判断见 6.30-6.32

二、结构冲突的解决



本图定位：结构冲突的典型场景与硬件解法

考点/易错

- 典型场景：取指与访存同时需要存储器；ALU 与地址计算同时需要运算器
- 解决：资源重复（指令 Cache 与数据 Cache 分离、增加加法器/端口）、资源分时、插入气泡
- 现代 CPU 的 L1 Cache 通常采用哈佛结构（指令/数据分离），就是为了消除取指与访存的结构冲突
- 易错：结构冲突不是数据相关；它是“资源不够”，不是“数据没算好”

补充

- 408 常问“指令 Cache 与数据 Cache 分离解决什么冲突”：结构冲突
- 资源重复会增加成本；资源分时会降低并行度

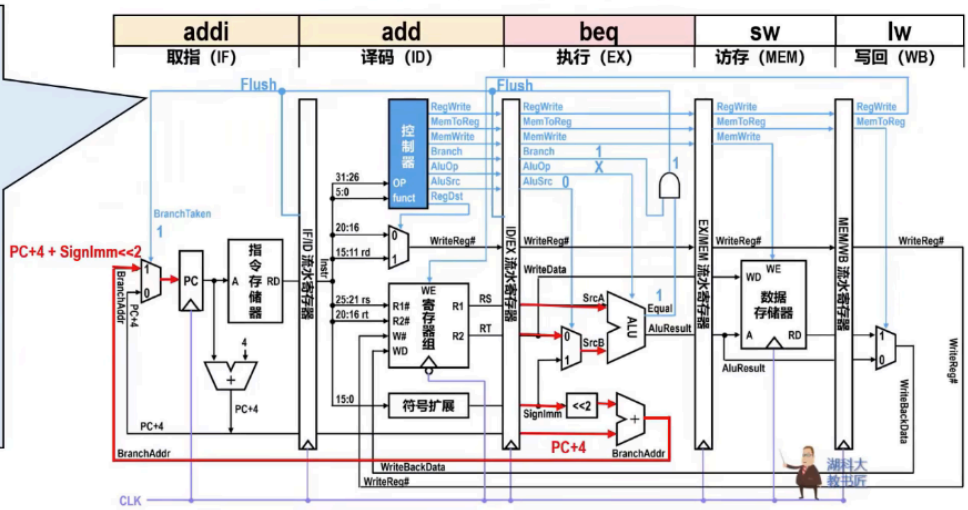
三、控制冲突的解决

控制冲突（冒险）与解决方案

■ **控制冲突**是指遇到**分支指令**或其他会**改变PC值的指令**时，在**分支指令之后载入流水线的后续指令可能因为分支跳转而不能进入执行（IF）阶段**，也称为**分支冲突**。

□ **解决控制冲突**的方法是，在执行程序分支跳转时**清除流水线中分支指令后续的若干条误取指令**。

- 为了实现指令清除，将BranchTaken信号作为流水线清空信号Flush直接连接IF/ID流水寄存器和ID/EX流水寄存器的同步清零控制端。
- 下一个时钟到来时：
  - beq指令进入MEM功能段，而IF功能段取出分支目标地址中的新指令。
  - IF/ID、ID/EX流水寄存器中锁存的数据和控制信号全部清零。

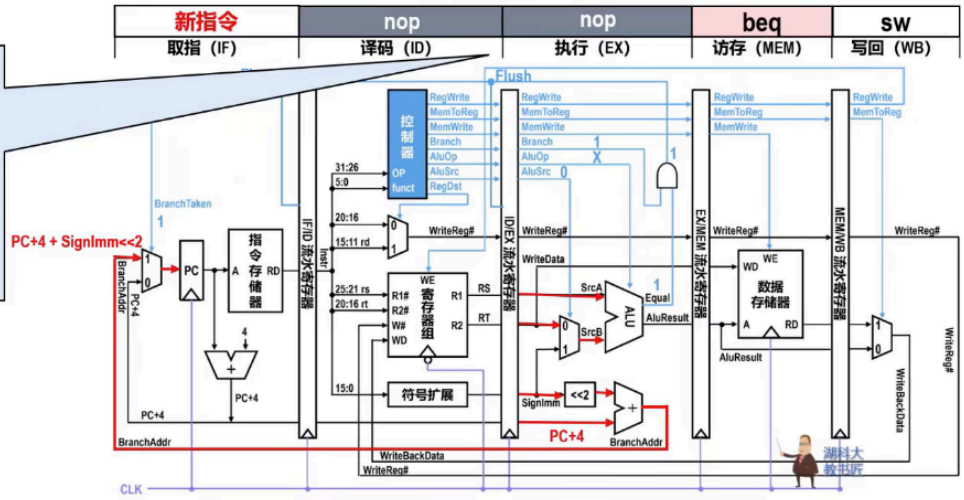


控制冲突（冒险）与解决方案

■ **控制冲突**是指遇到**分支指令**或其他会**改变PC值的指令**时，在**分支指令之后载入流水线的后续指令可能因为分支跳转而不能进入执行（IF）阶段**，也称为**分支冲突**。

□ **解决控制冲突**的方法是，在执行程序分支跳转时**清除流水线中分支指令后续的若干条误取指令**。

- **全零的MIPS指令**是 sll $0,$0,0，等同**空操作指令nop，不改变CPU状态。**
- 因此，在EX功能段和ID功能段中处理的指令变成了nop，误取入流水线的add指令和addi指令被成功清除。



控制冲突（冒险）与解决方案

```
1   beq $0, $0, 2        # 跳转到第4条指令
2   add $1, $2, $3       # 由于第1条指令而不会执行
3   addi $4, $5, 8       # 由于第1条指令而不会执行
4   bne $1, $2, 2        # 跳转到第7条指令（假设两个寄存器的内容不相等）
5   lw $5, 4($1)         # 由于第4条指令而不会执行
6   sw $6, 8($1)         # 由于第4条指令而不会执行
7   ……                   # ……
```

时钟周期	IF段	ID段	EX段	MEM段	WB段
1	beq				
2	add	beq			
3	addi	add	beq		
4	bne	nop	nop	beq	
5	lw	bne	nop	nop	beq
6	sw	lw	bne	nop	nop
7	第7条指令	nop	nop	bne	nop

- 分支指令（beq、bne）在EX段执行。
- 已经在流水线中ID段和IF段的两条指令必须被清空（Flush）成为两条空指令nop（气泡），因此会浪费两个时钟周期。从本例整体看，beq指令和bne指令各浪费两个时钟周期。
- 如果能够在ID段执行分支指令，则只浪费1个时钟周期。从本例整体看，beq指令和bne指令各浪费1个时钟周期。

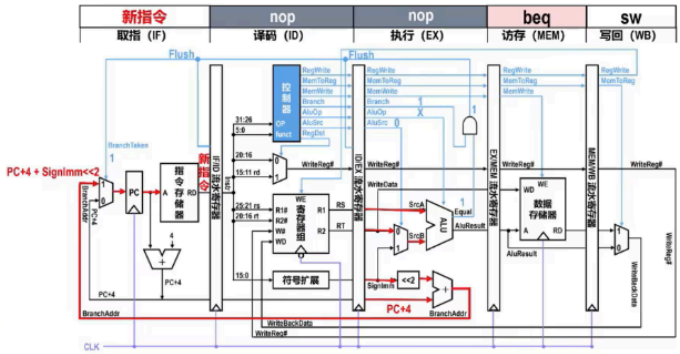


控制冲突（冒险）与解决方案

时钟周期	IF段	ID段	EX段	MEM段	WB段
1	beq				
2	add	beq			
3	addi	add	beq		
4	bne	nop	nop	beq	
5	lw	bne	nop	nop	beq
6	sw	lw	bne	nop	nop
7	第7条指令	nop	nop	bne	nop

整体减少2个时钟周期

时钟周期	IF段	ID段	EX段	MEM段	WB段
1	beq				
2	add	beq			
3	bne	nop	beq		
4	lw	bne	nop	beq	
5	第7条指令	nop	bne	nop	beq



- 提前到ID功能段执行，需要在ID功能段增加**比较器**和**地址加法器**。
- 现代MIPS处理器多在ID功能段执行分支指令，并配合**分支延迟槽技术**减少分支指令带来的流水线损失。



本图定位：分支/跳转让流水线“取错指令”的问题与解法

考点/易错

- 分支结果确定得越晚，流水线浪费越大；把分支判断提前到 ID 段可减少损失
- 解决：分支预测（静态/动态）、延迟槽、提前判断、预测错误时冲刷流水线
- 静态预测：总预测 taken/not taken 或按方向预测；动态预测：根据历史/计数器预测
- 预测错误要清除错误取入的指令（flush），并恢复正确 PC
- 延迟槽：在分支指令后安排一条无论分支是否成立都执行的指令
- 易错：分支预测不是“消除分支”；预测错误仍有代价
- 易错：控制冲突不是数据冲突；判断依据是“是否改变 PC/取指方向”

补充

- 408 常问“哪种方法对流水线性能影响最小/最大”：预测准确率越高，损失越小；延迟槽需要编译器配合
- 现代 CPU 用 BTB、方向预测器、返回地址栈等，但 408 只需掌握静态/动态预测与冲刷

四、易错辨析

- 结构冲突 = 资源争用；控制冲突 = 取指方向不确定；数据冲突 = 数据未就绪
- 指令/数据 Cache 分离解决结构冲突，不是数据冲突
- 分支预测解决控制冲突，不是数据冲突
- 延迟槽不是“延迟分支”，而是分支后固定执行的一条指令
- 预测错误必须冲刷；冲刷会浪费已取入的指令

五、知识关联

- 前置：6.27 指令流水线——概述、6.28 指令流水线——数据通路
- 后续：6.30 指令流水线——流水线冲突解决（数据冲突）、6.31 指令流水线——数据冲突的解决（插入气泡法）、6.32 指令流水线——数据冲突的解决（数据旁路法）
- 全章导航：6.0 导航与术语